LabScheduling — Architecture and Pipeline Map

Status: living document. Scope: the full system from raw data ingestion to Excel export, the solver core, the diagnostics layer, and the roadmap for scaling the engine beyond the university lab use case (exam scheduling, an AI university agent via API/MCP, other assignment/scheduling problems).

The core idea of the project: build a general **optimization and planning engine**. The current context is university lab scheduling, but the engine is meant to be reusable as an extension for similar constrained assignment problems.

---

1. The engine in one sentence

Given students, their course enrollments, professor teaching assignments, the official course timetable, room capacities and the academic calendar, the engine produces a **valid, balanced weekly lab schedule**: which group meets on which day / time block / room / week, who supervises it, and how many credits each professor spends — while flagging every incoherence instead of hiding it.

Guiding principle, preserved everywhere in the codebase:

> “Assignment is data; the system validates it, it does not decide it.”
> (Signal, do not fabricate.)

---

2. Optimization method (the honest answer)

The engine is **hybrid: heuristic group formation + CP-SAT week placement**.
It is NOT “hand everything to the solver”.

Phase	Who decides	What is decided
Group formation (heuristic)	`form_groups()`	For each lab group: its **day**, **time block**, **room**, and its **student members**. Respects free/busy slots, min/max group size, shared groups (Fisica/Quimica), program homogeneity for first year, professor-busy slots, and a soft Friday anti-bottleneck penalty.
Week placement (CP-SAT)	`solve()`	The only free variable is **which academic week** each session runs in. The solver spreads sessions, avoids clashes (C1/C4), enforces chronological order (C5), and minimizes soft spacing penalties.
Professor assignment (post-hoc)	`lab_professor_assignment.py`	After a schedule exists, the responsible professor per group is derived from the official P credits (1 P credit = 5 sessions), proportionally. Persisted into the schedule (`professor` column).

Consequence to keep in mind: **the CP-SAT model does not choose the slot or the room** — those are fixed upstream by the heuristic. So an "infeasible" CP-SAT result is almost always a **week-capacity** problem (too many sessions forced into the same room/slot across too few available weeks), not a slot-assignment problem. This is exactly what the infeasibility diagnostic reports.

---

3. Mapping to the whiteboard architecture (`architecture.jpg`)

The whiteboard sketch defines the intended data flow. The code follows it:

Whiteboard node	Code responsibility	Notes
Students -> enrollment	`identify_students()`	Builds subject -> student sets from the master schedule.
Course schedule -> Student schedule	`build_individual_timetables()`	Per-student busy/free slots derived from their course timetable.
Student schedule -> busy / free slots	`student_busy`, `student_subject_slots`	30 possible slots (5 days x 6 blocks); occupied slots removed.
Professors -> teaching assignments (normal class)	`prepare_professor_constraints()`, `build_professor_busy()`	Professor busy slots from their normal teaching load.
Course schedule -> Professor schedule -> busy / free	`build_subject_professor_busy()`	A group of a subject cannot be created on a slot where a professor of that subject is busy (soft-relaxable).
Professor lab teaching assignment: which labs? how many credits / groups?	`lab_professor_assignment.py`, `professor_credits.py`	Which labs, how many P credits, how many groups per professor — derived from the official “Asignacion docente”.
Coincidencias / Gaps	soft objective terms in `solve()` + `reliability_metrics.py`	Spacing/gap deviation minimized; balance and health scored post-hoc.

The whiteboard’s “busy/free” duality is the backbone: **every actor (student, professor, room) has a busy set, and the complement is the free set** the engine schedules into.

---

4. End-to-end pipeline map

Entry point: `pipeline.py :: main()` -> `run_pipeline(df)`. Each stage reads and writes explicit artifacts, so any stage can be inspected or replaced.

```
                    data_clean/master_schedule.csv
                                  |
                                  v
[1] load_and_prepare(df)                      normalize columns, types
                                  |
                                  v
[2] identify_students(df)                      subject -> {student ids}
                                  |
                                  v
[3] build_individual_timetables(df, ...)       student_busy, student_subject_slots
                                  |
                                  v
[4] prepare_professor_constraints(df)          subject_professor_busy,
    build_student_program_lookup(df)           subject_block_penalty,
                                                professors_of_subject
                                  |
                                  v
=============== HEURISTIC ===============
[5] form_groups(...)                           all_groups: each group has a
                                                FIXED (day, block, room) and its
                                                student members
                                  |
                                  v
[6] run_data_quality_checks(...)  (data_quality.py, non-blocking)
    audit_teacher_max_days(...)
    verify_availability_constraints(...)       config/availability_verification.json
                                  |
                                  v
=============== CP-SAT SOLVER ===============
[7] solve(all_groups)                          per semester:
      - week_vars: IntVar per session (domain = valid weeks)
      - C1: no two sessions of same SUBJECT in same (week, day, block)
      - C4: no two sessions in same ROOM in same (week, day, block)
      - C4-reserved: soft penalty for externally reserved room-slots
      - C5: sessions of a group are chronologically ordered
      - soft objective: first-session anchor, last-session anchor,
        even gap spacing, parity alternation, Friday cap
      - status -> record_solver_run() -> reports/solver_stats.json
      - if INFEASIBLE: diagnose_infeasibility() + automatic recovery
        (drop over-saturated groups, re-solve)
                                  |
                                  v
    results_df (semester, subject, grupo, session, week, day, time_block, room)
                                  |
                                  v
=============== POST-PROCESSING ===============
[8] lab_professor_assignment.assign_professors_to_schedule_df(results_df)
      -> results_df['professor']   (P0.1: professor persisted in schedule)
                                  |
                                  v
[9] kpi_report.generate_kpi_report(...)        reports/kpi_report.{json,txt}
                                  |
                                  v
[10] generate_outputs(...)                     outputs/optimization/*.csv + .xlsx
     analyze(results_df)                        (Grupo de practicas, Vista profesor,
                                                 Teacher View, quality sheets)
                                  |
                                  v
[11] run_daniel_format_generation()            Distribucion_Practicas_*.xlsx
     create_auto_snapshot()                     versions/<name>/
```

Stage responsibility table

#	Stage	Function (file)	Reads	Writes
1	Load & prepare	`load_and_prepare` (pipeline.py)	master_schedule.csv	in-memory df
2	Identify students	`identify_students`	df	subject_students
3	Student timetables	`build_individual_timetables`	df, subject_students	student_busy, student_subject_slots
4	Professor constraints	`prepare_professor_constraints`, `build_professor_busy`	df, Asignacion	subject_professor_busy, professors_of_subject
5	Group formation (heuristic)	`form_groups`	all of the above	all_groups (day/block/room fixed)
6	Data quality / audits	`data_quality.py`, `audit_teacher_max_days`, `verify_availability_constraints`	df, all_groups	data_quality_report.*, availability_verification.json
7	CP-SAT solve	`solve`	all_groups	results_df, solver_stats.json, infeasibility_S*.txt
8	Professor assignment	`lab_professor_assignment.py`	results_df, Asignacion	results_df[‘professor’]
9	KPI report	`kpi_report.py`	results_df, all_groups, solver_runs	kpi_report.{json,txt}
10	Excel/CSV export	`generate_outputs`, `excel_generator_core.py`	results_df	outputs/optimization/*.{csv,xlsx}
11		`run_daniel_format_generation`,	outputs	

#	Stage	Function (file)	Reads	Writes
	Daniel format + snapshot	`create_auto_snapshot`		Distribucion_Practicas_*.xlsx, versions/

---

5. The CP-SAT model in detail

Built inside `solve()`, one model per semester.

Variables

- `week_vars[session_id]`: integer variable, domain = the valid weeks for that session (min_week..max_week minus holidays for that weekday).

Hard constraints

- **C1** (subject clash): for sessions of the same subject on the same (day, block), all weeks must differ.
- **C4** (room clash): for sessions in the same room on the same (day, block), all weeks must differ.
- **C5** (chronological order): within a group, session k+1 runs strictly after session k.

Soft constraints (objective, minimized)

- **C4-reserved**: heavy penalty when a session lands on an externally reserved room-slot (allowed, but strongly discouraged).
- **First-session anchor**: penalize starting late.
- **Last-session anchor**: penalize finishing early.
- **Gap spacing**: penalize deviation from the ideal even spacing between sessions.
- **Parity alternation**: alternate parallel groups across even/odd weeks.
- **Friday cap**: constant + escalating penalty above a soft session cap on Fridays (anti-bottleneck), never a hard ban.

> **Phase 2 note — configurable weights.** The first-anchor, last-anchor, spacing and parity penalties now read their weights and on/off state from `config/solver_constraints.yaml` via `solver_config.py`. When the file is absent or invalid, the historical defaults (100 / 100 / 200 / 50) are applied, so behaviour is unchanged. See section 10.

Warm start

`add_week_hints()` injects an evenly-spread week assignment as a solver hint to speed convergence.

---

6. Solver status semantics (why OPTIMAL / FEASIBLE / INFEASIBLE)

`record_solver_run()` writes one entry per solve into `reports/solver_stats.json` with: status, objective, best_bound, gap, wall_time_s, n_sessions, n_hints, recovered.

Status	Meaning	Typical cause	Recommended reaction
OPTIMAL	Proven best solution; gap ≈ 0	Model well constrained, time sufficient	Ship it.
FEASIBLE	A valid solution found, but not proven optimal (gap > 0)	Time limit hit before proof, or hard combinatorics	Usually fine to ship; increase time limit or relative gap to tighten if desired.
INFEASIBLE	No valid week assignment exists	Physical capacity: too many sessions forced into the same room/slot across too few available weeks	Read `infeasibility_S*.txt` bottlenecks; open a slot/room, widen the [min_week, max_week] window, or reduce parallel groups. Automatic recovery drops the over-saturated groups and re-solves.
UNKNOWN	Solver stopped without conclusion	Time limit with no solution	Increase time limit; inspect bottlenecks.

Automatic recovery

When a semester is INFEASIBLE, `solve()` does not give up:
1. `diagnose_infeasibility()` scans room-slot and subject-slot groups for over-saturation (needed sessions > available weeks) and writes a readable report.
2. The over-saturated groups are dropped by priority (overflow > recovered > alt_room), the model is rebuilt on the remaining sessions and re-solved.
3. Dropped groups are flagged `_solver_dropped` and surfaced downstream; the run is recorded with `recovered=True`.

This is the “signal, do not hide” contract: an infeasibility is always explained and, when possible, recovered — never silently swallowed.

Unplaced students

`diagnose_unplaced_students()` explains, per enrolled-but-unplaced student, why: either a total timetable conflict (no common slot with any group), saturated compatible slots (room/group capacity reached), or a cohort/program constraint. Written to `reports/unplaced_students.json`.

---

7. Module map (current monolith)

File	Role
`pipeline.py`	Orchestrator: ingestion, heuristic, CP-SAT, recovery, exports. Core of the engine.
`form_groups` (in pipeline.py)	Heuristic group formation.
`solve` (in pipeline.py)	CP-SAT week-placement model + recovery.
`lab_professor_assignment.py`	Per-group responsible professor from P credits.
`professor_credits.py`	P/T credit parsing and budgets from Asignacion docente.
`lab_constants.py`	Single source of truth for `CREDIT_TO_SESSIONS = 5`.
`data_quality.py`	Non-blocking data-integrity checks (join reconciliation, anti-leak).
`kpi_report.py`	Objective KPIs (placement, balance, rooms, solver) per run.
`validation_credits.py`	Credit rule validation.
`reliability_metrics.py`	0-100 health score and reliability signals.
`monitoring.py`	Streamlit “control tower” page: pure collectors + render.
`excel_generator_core.py`	Excel workbook generation (Grupo de practicas, Vista profesor, Teacher View, quality sheets).
`app.py`	Streamlit application shell and page routing.

Data artifacts

Inputs (read):
- `data_clean/master_schedule.csv` — the consolidated source of truth.
- Asignacion docente file — official professor credits.
- Aulario / Alumnos sources — rooms and students.
- `config/user_config.json` — user overrides and teacher rules.

Outputs (written):
- `outputs/optimization/optimized_schedule_v5.{csv,xlsx}` — the schedule.
- `outputs/optimization/group_composition.csv` — groups and members.
- `reports/solver_stats.json` — one entry per solve.
- `reports/unplaced_students.json` — unplaced diagnostics.
- `reports/infeasibility_S*.txt` — infeasibility bottlenecks.
- `reports/kpi_report.{json,txt}`, `reports/data_quality_report.*` — quality.
- `versions/<name>/` — auto snapshots.

---

8. Architecture assessment and scaling roadmap

The current design is a **modular monolith**: a single Streamlit app plus a procedural pipeline, with the business logic already split into focused modules (`professor_credits`, `lab_professor_assignment`, `data_quality`, `kpi_report`, `reliability_metrics`, `monitoring`). This is the right choice for the current scale and the desktop `.exe` distribution: simple to build, ship and debug.

To scale toward an enterprise / multi-tenant engine (exam scheduling, an AI university agent, other institutions), the recommended evolution is incremental, never a rewrite (consistent with `FUNCTIONAL_COMPARISON.md`):

1. **Isolate the solver core** behind a stable, framework-free interface: `build_model(problem) -> model`, `solve(model) -> solution`, `diagnose(model) -> report`. This is the seam that makes every future use case (labs, exams) a different `problem` input to the same core.
2. **Introduce a problem schema** (a typed description of actors, slots, constraints, objectives) so new domains are configuration, not new code. This is the foundation for the “configurable soft/hard constraints” request.
3. **Expose an API layer** (FastAPI) over the core, then an **MCP server** that maps tools to the same endpoints, for the AI-agent integration.
4. **Add a job queue** (e.g. RQ/Celery) for long solves so the API stays responsive; persist runs in a real store rather than loose JSON files.
5. **Package multi-platform** (Docker image + hosted web app) so macOS/Linux users are not blocked by the Windows-only `.exe`.

Only when the problem schema and multiple domains genuinely exist does a fuller layered (“clean”) architecture pay for itself. Until then, the modular monolith plus a cleanly isolated solver core is the pragmatic, maintainable path.

---

9. Observability contract

Everything the engine does is meant to be inspectable, not hidden:
- Every solve is journaled (`solver_stats.json`), with status, gap and time.
- Every infeasibility is explained (`infeasibility_S*.txt`) and, when possible, recovered.
- Every unplaced student has a verdict (`unplaced_students.json`).
- Data quality, KPIs and a health score are produced on every run.
- The `monitoring.py` control-tower page aggregates all of the above into one read-only view, including a Solver Diagnostics panel that translates raw solver status into a plain-language “why this result and what to do next”.

This observability is the QA backbone that lets the same engine be trusted in production and extended to new use cases.

---

10. Phase 2 components (configurable constraints + infeasibility simulation)

Phase 2 adds two additive, read-only-by-default capabilities. No validated solver logic or data-processing stage is modified; the solver only reads weights that default to the historical values.

10.1 Configurable soft constraints

Piece	File	Role
Schema + loader	`solver_config.py`	Validated load/save of soft-constraint weights and on/off flags; 3 presets (Strict / Balanced / Relaxed); defaults reproduce the historical weights.
Config file	`config/solver_constraints.yaml`	Human-editable weights, flags and preset reference. Missing/invalid -> Balanced defaults.
Solver hook	`pipeline.solve()`	Loads config once, applies effective weights to the objective terms, and appends a `constraint_config` summary to each `solver_stats.json` entry. A disabled constraint contributes weight 0, i.e. the term is skipped.
UI	`pages/4_Configuration_Solveur.py`	Profile selector, per-constraint toggles + weight sliders, live preview, extreme-config warnings, save/reset with confirmation.

The four tunable soft constraints are `semester_anchor_first`, `semester_anchor_last`, `spacing`, `parity`. Hard constraints (C1/C4/C5) and the reserved-slot penalty are intentionally NOT exposed (they are correctness, not preference).

10.2 Infeasibility simulation (What-If)

Piece	File	Role
Engine	`simulation_engine.py`	Pure, side-effect-free capacity/bottleneck model plus an optional real CP-SAT feasibility dry-run. Never writes to `reports/` / `outputs/`.
UI	`pages/5_Simulateur_Infaisabilite.py`	Three sections: (1) exclude groups, (2) add room/time-slot capacity, (3) automatic suggestions from detected bottlenecks.

Engine entry points:
- `simulate_without_groups(sessions, group_ids)` — recompute bottlenecks with groups removed; reports overflow reduction and affected students.
- `simulate_with_extra_capacity(sessions, extra_slots)` — recompute with extra weeks added to given `(resource, day, block)` slots.
- `suggest_actions(sessions)` — rank groups to drop / resources to add.
- `dry_run_feasibility(sessions)` — build C1/C4/C5 and solve for feasibility only.
- Read-only loaders parse `reports/unplaced_students.json`, `reports/solver_stats.json` and `reports/infeasibility_S*.txt`.

Every simulation returns a `before` / `after` / `diff` structure so the UI can show the metric delta and whether a scenario would become feasible.

See `PHASE2_FEATURES.md` for usage details.